Security Audit - Vaults

conducted by Neodyme AG

Lead Auditor:	Jasper Slusallek
Second Auditor:	Mathias Scherer
Administrative Lead:	Thomas Lambertz

January 23, 2025

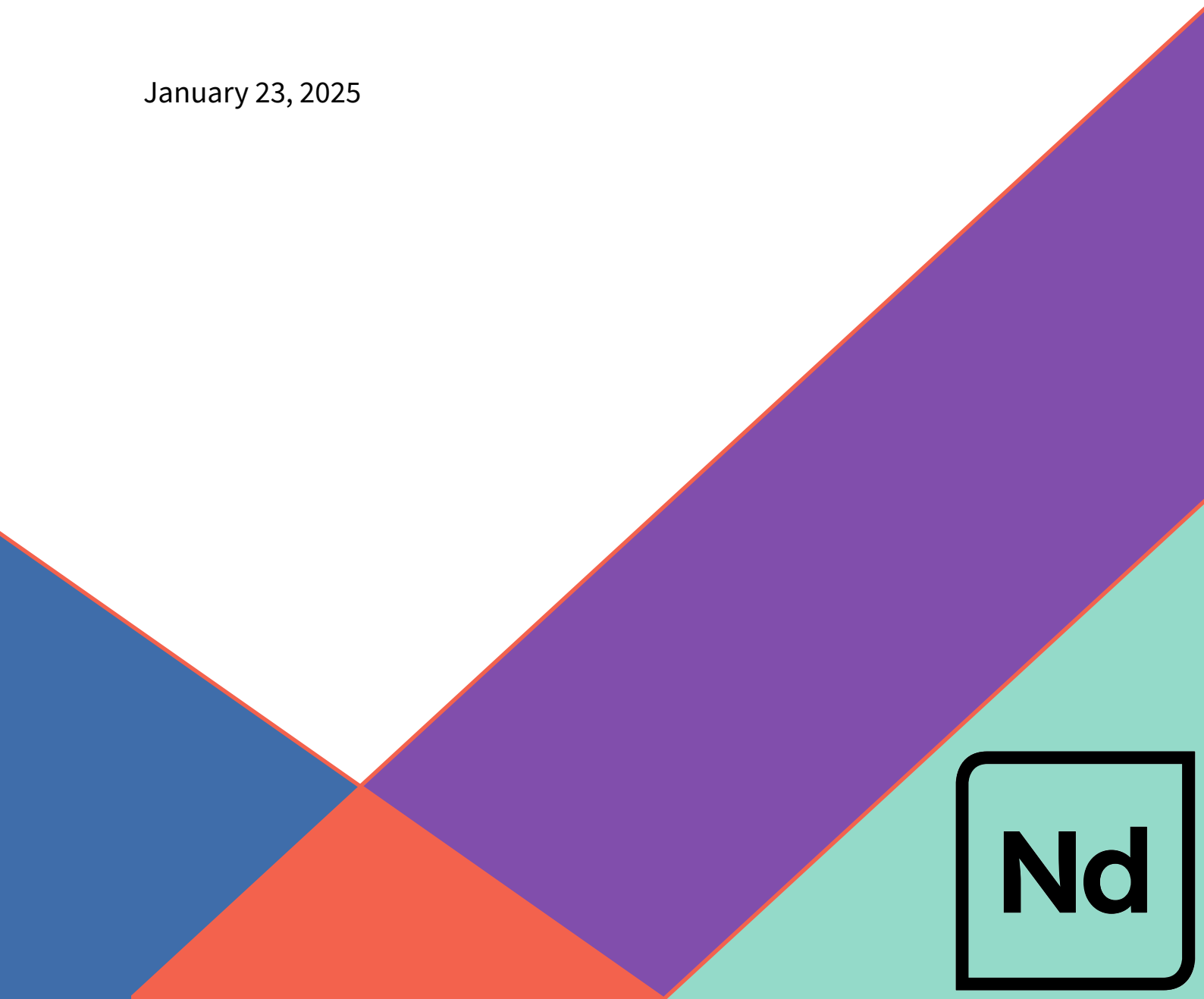


Table of Contents

1 Executive Summary **3**

2 Introduction **4**

Summary of Findings 4

3 Scope **5**

4 Project Overview **6**

Functionality 6

On-Chain Data and Accounts 8

Instructions 10

Authority Structure and Off-Chain Components 13

5 Findings **15**

[ND-DV-HI-01] Drift's delegate functionality is insufficiently constrained 16

[ND-DV-MD-01] Incorrect accounting in withdraw request cancellation instructions 18

[ND-DV-IN-01] Incorrect account check comments 21

[ND-DV-IN-02] Unnecessary unconstrained accounts in RequestWithdraw 23

[ND-DV-IN-03] Vault Token Accounts: Naming and Seeds 25

Appendices

A About Neodyme **26**

B Methodology **27**

Select Common Vulnerabilities 27

C Vulnerability Severity Rating **29**

1 | Executive Summary

Neodyme audited **Drift's** on-chain Drift Vaults program from November 2024 until December 2024.

The scope of this audit included the implementation security of the contract itself, security against economic attacks as well as an analysis of its interactions with the Drift Protocol v2 contract.

The auditors found that Drift's Vaults comprised a clean design. According to Neodymes Rating Classification, **2 security relevant** and **3 informational** were found. The number of findings identified throughout the audit, grouped by severity, can be seen in Figure 1.

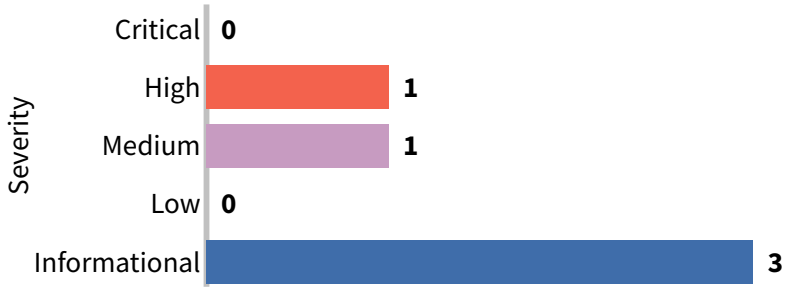


Figure 1: Overview of Findings

The auditors reported all findings to the Drift developers, who addressed them promptly. The security fixes were verified for completeness by Neodyme.

In addition to these findings, Neodyme delivered the Drift team a list of nit-picks and additional notes that are not part of this report.

2 | Introduction

From November 2024 until December 2024, Drift engaged Neodyme to do a detailed security analysis of their Vaults program. Two senior security researchers from Neodyme, Mathias Scherer and Jasper Slusallek, conducted independent full audits of the contract between the 20th of November 2024 and the 23rd of December 2024. Both auditors have a track record of finding critical and other vulnerabilities in Solana programs, and have audited many similar programs. Jasper Slusallek was additionally part of Neodyme's review of the Drift Protocol v2 main contract and laid a focus on any security concerns arising from interactions between the contracts.

The audit focused on the contract's technical security. In the following sections, we present our findings and discuss worst-case scenarios for authority compromise, and provide some general notes for considerations that may be useful in the future.

Drift's team always responded **competently** to all findings. The low volume of findings speaks to the significant effort which was evidently put into the design of the contract. The overall architecture of the program is **clean and coherent**. The interactions with Drift Protocol v2 are cleanly separated from the internal contract logic and kept to a minimum such that **no particularly complicated interactions** arise.

The authority structure of the contract was also kept simple and intuitive. We discuss it in more detail in a later section.

Summary of Findings

All found issues were **quickly remediated**. In total, the audit revealed:

0 critical • **1** high-severity • **1** medium-severity • **0** low-severity • **3** informational

issues.

The high-severity finding arose due to an interaction with Drift Protocol v2, where an assumption on the constraints of an authority was made. These constraints could be circumvented via malicious actions within Drift Protocol v2, resulting in a full loss-of-funds vulnerability in Vaults, which can be triggered if a vault ever enters liquidation.

3 | Scope

The contract audit's scope comprised of three major components:

- Primarily, the **Implementation** security of the contract's source code
- Additionally, security of the **overall design**
- Additionally, resilience against **economic attacks**
- Finally, correctness and security of the **interactions with Drift Protocol v2**

Neodyme considers the source code, located at https://github.com/drift-labs/drift-vaults, to be in scope of this audit. Third-party dependencies are not in scope, however note that Drift Vaults only relies on the Anchor library, drift, bytemuck, and drift-macros.

During the audit, minor changes and fixes were made by Drift, which the auditors also reviewed in-depth.

Relevant source code revisions are:

- `e203036f416c5f2a72106910be9868b735acbe0f` · Start of the audit
- `31f0656d5f0d2aa4cc2e4ea8dce01d08ef28727d` · Last reviewed revision

4 | Project Overview

This section briefly outlines Vaults's functionality, design, and architecture, followed by a detailed discussion of all related authorities.

Functionality

Drift Vaults provides the functionality for so-called managers to open Vaults. These are managed funds which employ certain investment strategies within Drift Protocol v2.

General overview

Each vault manages its assets in a user account it maintains with Drift Protocol v2.

Users of the Vaults program can deposit into a vault in exchange for shares representing their deposits. Their deposits are immediately sent into the corresponding Drift Protocol v2 account and can be deployed therein by the manager according to their advertised investment strategy. The manager cannot withdraw the deposited assets, only trade with them.

Users can trade in their shares at any point in time in exchange for a proportional share of the total current valuation in the Drift account.

Technical implementation of the manager restrictions

The restriction that the manager cannot withdraw the vault's funds but can trade them is implemented via a functionality in Drift Protocol v2.

Namely, the manager is set as the Drift account's "delegate". This authority is restricted exactly in this way.

User shares and tokenization

User's shares are tracked in `VaultDepositor` accounts which track the shares held internally.

A user can decide to tokenize their shares such that they can be freely transferred between token accounts. This functionality has to be enabled by the manager by creating a `TokenizedVaultDepositor` account. Shares are internally transferred from the user account before the tokenized shares are minted. Tokenized shares can be redeemed back into internally tracked shares.

Rebasing

As share amounts may grow over time, shares can be rebased – that is, all amounts of the shares may be divided by a power of 10.

To do this, all accounts that hold share amounts internally track the log10 base of the share that they are tracking. If the vault decides to rebase, all relevant accounts are then lazily rebased as soon as the discrepancy in the base is noticed.

Withdrawal Requests

If a manager, the protocol fee withdrawal authority, or a user wishes to withdraw funds, they must open a withdraw request and wait for a set redemption period (up to 90 days), after which they can fulfil the withdraw request

However, the withdraw request stores the value of the shares to be burned at the time of withdrawal request. When the withdrawal request is fulfilled, all shares are burned, but the payout is the minimum of this old value and the current value of the shares. This way, if the shares appreciate between request and fulfilment, the withdrawer forfeits these gains.

Notably, gains since request creation are also forfeited if the withdrawal request is cancelled. Any gains forfeited are distributed among the other shareholders.

Liquidation

If a user decides to withdraw from a vault by trading in their shares, the Drift v2 account managed by the vault must have sufficient free collateral for the withdrawal to succeed. If not, the vault enters liquidation.

In the original implementation, this was implemented such that the user is designated as the delegate of the Drift v2 account instead of the manager for a short period of time. Additionally, the reduce-only flag is set in Drift, which means that all trades must be risk-reducing (to free up the needed collateral).

However, we identified ways to circumvent these restrictions, see the high-severity finding.

As a result, Drift introduced a permissioned liquidator instead, who will free any collateral as necessary to fulfil relevant withdraw requests.

Fees

Finally, the program has two types of fees:

1. The management fee (paid directly to the manager responsible for managing the vault).
2. The protocol fee

Additionally, the program includes a profit-sharing feature for the manager and the vault protocol. Once the vault has been set up, the fees and profit shares cannot be increased; they can only be decreased. Note that the vault protocol must be defined during the vault initialization and cannot be added after the vault has been initialized.

On-Chain Data and Accounts

The Vaults contract needs to keep track of the vault, their depositors, and other important information for the contract to work. This is done by defining four different account types outlined here.

The **`Vault`** account is the main account used by the vault itself. It is seeded by is seeded by the seeds `"vault" + vault_name`. The most relevant information it keeps track of is:

- name of the vault
- vault manager key
- info on the Drift v2 account managed by the vault, as well as the relevant Drift spot market
- configuration values, i.e:
  - delegate (who is set as the Drift Protocol v2 delegate on the vault account)
  - permissioned liquidator (if not in liquidation, set to `PubKey::default()`)
  - max tokens allowed to be deposited
  - management fee and profit ratio
  - whether the vault is permissioned or not
  - hurdle rate (currently unimplemented)
  - redeem period the users have to wait for a withdraw request to be redeemable
- accounting values, i.e:
  - sum of shares owned by the users and protocol
  - shares base (see Rebasing)
  - total sum of currently pending withdrawals
  - last fee update timestamp
- liquidation status of the vault
- statistics on deposits, withdraws, total shares issued etc

The **`VaultProtocol`** account is used to give a cut of the fees and profits to the protocol enabling the vaults strategy. This account is seeded from `"vault_protocol" + vaul.key` and (most relevantly) holds the following information:

- profit and fee share
- protocol fee
- total withdraws for the protocol
- protocol total fee
- protocol total profit share

Each user has a **`VaultDepositor`** account unique for each vault they deposit into. Its responsibility is to keep track of user deposits and is seeded from `"vault_depositor" + vault.key() + authority.key()`. The key data it holds is:

- vault to which it belongs
- associated authority
- shares held and their base
- last withdrawal request made (including time of request)
- statistics on deposits, withdrawals and fees

The last account used is the **`TokenizedVaultDepositor`**, which allows a user to tokenize the shares from the `VaultDepositor`. The `TokenizedVaultDepositor` is seeded from `"tokenized_vault_depositor" + vault.key() + vault.shares_base`. As seen by the seeds used for the `TokenizedVaultDepositor` account, this account is bound to the vault shares base, resulting in the manager being required to create a new `TokenizedVaultDepositor` in case the vault got rebased to a new divisor. The `TokenizedVaultDepositor` keeps track of the following key data:

- vault to which it belongs
- mint it manages
- total shares held (i.e. tokenized)
- statistics on deposits, withdrawals and fees

Instructions

The contract has a total of 32 instructions, which we briefly summarize here. The instructions are grouped by purpose.

Initialization & Updating

Instruction	Category	Summary
InitializeTokenizedVaultDepositor	Manager	Initializes a new TokenizedVaultDepositor.
InitializeVaultDepositor	Permissionless / Manager	Initializes a new VaultDepositor. This action can only be done by the manager if the vault is permissioned.
InitializeVaultWithProtocol	Permissionless	Initializes a new vault with a vault protocol.
InitializeVault	Permissionless	Initializes a new vault without a vault protocol.
UpdateDelegate	Manager	Sets a new vault delegate if the vault is not in liquidation.
UpdateMarginTradingEnabled	Manager	En-/Disables margin trading for the vault if not in liquidation.
UpdateVaultProtocol	VaultProtocol	Updates the protocol fee and profit share, has to be lower than the current.
UpdateVault	Manager	Updates vault settings. Fees, periods and rates can only be lowered.

Management & Protocol

Instruction	Category	Summary
ManagerDeposit	Manager	Deposits the given amount of tokens into the vault.
ManagerRequestWithdraw	Manager	Request the given amount to be withdrawn.
ManagerCancelWithdrawRequest	Manager	Cancel the current withdraw request.
ManagerWithdraw	Manager	Withdraw the previously requested shares. Only available after the vault redemption period has passed.
ApplyProfitShare	Manager, VaultDelegate	Applies the profit share for the given VaultDepositor.

Instruction	Category	Summary
ForceWithdraw	Manager, VaultDelegate	Forcefully withdraws the pending withdraw request of the given VaultDepositor. Only available after the vault redemption period has passed.
ProtocolCancelWithdrawRequest	Protocol	Cancel the current withdraw request.
ProtocolRequestWithdraw	Protocol	Request the given amount to be withdrawn.
ProtocolWithdraw	Protocol	Withdraw the previously requested shares. Only available after the vault redemption period has passed.

Insurance Fund

Instruction	Category	Summary
InitializeInsuranceFundStake	Manager	Initializes the drift insurance funds for the given vault and market index.
AddInsuranceFundStake	Manager	Adds stake into the drift insurance fund.
RequestRemoveInsuranceFundStake	Manager	Requests the given amount to be withdrawn from the insurance fund.
CancelRequestRemoveInsuranceFundStake	Manager	Cancels the previous request to withdraw stake from the insurance fund.
RemoveInsuranceFundStake	Manager	Removes the previously requested amount from the insurance fund.

User Asset Management

Instruction	Category	Summary
Deposit	User	Deposits the given amount of tokens into the vault.
RequestWithdraw	User	Request the given amount to be withdrawn.
CancelWithdrawRequest	User	Cancel the current withdraw request.
Withdraw	User	Withdraw the previously requested shares. Only available after the vault redemption period has passed.
TokenizeShares	User	Tokenizes the given amount with the current `TokenizedVaultDepositor`.

Instruction	Category	Summary
RedeemTokens	User	Redeems tokenized shares back into the `VaultDepositor`.

Other

Instruction	Category	Summary
ApplyRebaseTokenizedDepositor	Permissionless	Rebases the TokenizedVaultDepositor.
ApplyRebase	Permissionless	Rebases the given VaultDepositor to the vault share base.
Liquidate	User	Set themselves as the current liquidation delegate for the vault if the current withdrawal request cannot be fulfilled by the vault.
ResetDelegate	Permissionless	Reset the liquidation delegate if the time for the liquidation has passed.

Authority Structure and Off-Chain Components

In Drift Vaults, apart from user authorities, the following privileged authorities exist: Protocol signer, vault manager, vault delegate, liquidation delegate and upgrade authority. Finally, the upgrade authority of the contract also exists.

Of these, all but the Protocol signer are able to withdraw all funds from a Vault. In the sections below, we detail why and how.

Protocol Signer

The protocol signer (or simple `protocol`) is stored in the `VaultProtocol` account and can change the configuration parameters stored therein – meaning the protocol fee and profit share. However, these can only be lowered after initialization.

They can also withdraw any protocol funds via withdraw requests.

This, along with the fact that the fee and profit share can only be lowered, means a compromise of this authority affects only the protocol funds, not user or manager funds.

Vault Manager

The vault manager can update the delegate of the Drift v2 account of the vault. As such, via the process described in ND-DV-HI-01, they can theoretically withdraw all vault funds.

They can also update vault parameters, namely management fee and profit share (downward only), deposit restrictions, withdraw request waiting time, the permissioned flag and the hurdle rate (currently unused). Relatedly, they can enable and disable margin trading in the vault's Drift v2 account.

They may apply the profit share to any vault depositor, such that any outstanding profit shares of that vault depositor are applied to the protocol and manager. Of course, they can also withdraw any manager funds via withdraw requests.

Less relevantly, they have powers like depositing and withdrawing from the Drift v2 insurance fund, they can force any user withdraw request to be completed and sent to the user's ATA.

Obviously, due to the risks in the first paragraph, this authority should be heavily safeguarded. However, this is up to the organization providing the vault.

Vault Delegate

This is the authority to which the "delegate" field of the Drift v2 account is set. Due to the behaviour described in the section on the Vault Manager above, this authority can also theoretically withdraw all funds and should thus be heavily safeguarded.

They can also apply the profit shares or trigger a forced withdrawal, like the vault manager.

Liquidation Delegate

This is the authority to which the “delegate” field of the drift v2 account is set if a liquidation occurs.

Similarly to the Vault manager and Vault delegate, this means that it can theoretically withdraw all funds if a liquidation event occurs.

Upgrade Authority

As with any contract, the upgrade authority has complete control over the program and the funds it controls. It is, hence, one of the most critical components of the security of the protocol. By maliciously upgrading the contract, the upgrade authority can irreversibly transfer control of all funds to itself.

Drift is aware of this and has put considerable effort into making the upgrade authority safe. They have established a 3-out-of-5 multisig as its upgrade authority. The on-chain multisig is managed by a non-upgradable deployment of Squads V4 and can be viewed here.

5 | Findings

This section outlines all of our findings. They are classified into one of five severity levels, detailed in Appendix C. In addition to these findings, Neodyme delivered the Drift team a list of nit-picks and additional notes which are not part of this report.

All findings are listed in Table 6 and further described in the following sections.

Identifier	Name	Severity	Status
ND-DV-HI-01	Drift's delegate functionality is insufficiently constrained	HIGH	Resolved
ND-DV-MD-01	Incorrect accounting in withdraw request cancellation instructions	MEDIUM	Resolved
ND-DV-IN-01	Incorrect account check comments	INFORMATIONAL	Resolved
ND-DV-IN-02	Unnecessary unconstrained accounts in RequestWithdraw	INFORMATIONAL	Resolved
ND-DV-IN-03	Vault Token Accounts: Naming and Seeds	INFORMATIONAL	Acknowledged

Table 6: Findings

[ND-DV-HI-01] Drift's delegate functionality is insufficiently constrained

Severity	Impact	Affected Component	Status
HIGH	Loss of funds	Liquidation	Resolved

The following is an oversight in Drift v2 which, however, has major consequences in Drift Vaults. Specifically, the assumption that the Vaults contract places on Drift's delegate functionality is incorrect.

The contract assumes that Drift's delegate functionality constrains the delegate to only trading on behalf of the user. It assumes that the delegate cannot withdraw. If the delegate could withdraw, the following would be true:

1. During liquidation, any user could drain the Vault, since they are assigned to be the delegate of the Vault's `drift_user`
2. The manager could rug pull at any point.

Justification for 1: If a user should be able to trigger a matured withdraw request but cannot (due to either the initial margin requirement or a margin trading being violated if it were to be executed), the Vault enters liquidation. This works by setting the delegate of the Vault's `drift_user` to the user who is trying to withdraw [1]. This `drift_user` holds all of the vault's funds. The delegate can close all liabilities and drain the freed collateral.

Justification for 2: The manager sets the delegate for the Vault's `drift_user` in order to be able to trade its funds.

However, the delegate functionality in Drift is not airtight. While the delegate cannot call the withdraw function, they can subvert this withdrawal restriction via the methods explained below.

Note that liquidation may be triggerable by an attacker by e.g. depositing a large amount, waiting until the manager has deployed it such that the free collateral is insufficient for the withdrawal, and then withdrawing everything to trigger liquidation.

Method A: Flash Loans

The delegate can siphon off funds during flash loans (swaps). This was already mentioned in our audit discussions for Drift v2.

Basically, the flash loan instruction allows siphoning off funds during the swap as long as the swap is within a fairly generous price band (minimum of 10% of 5-min twap and within initial margin ratio of current oracle price, see [2]). To provide a bit more detail on this, here's how the transaction would look like:

1. Swapbegin instruction, where the swap will happen from token S to token T: The delegate (i.e. Vaults user) authorizes $X worth of tokens of type S to be transferred to a temporary account A.

2. Jupiter swap 1: The delegate swaps, say, 95% of the S-tokens into T-tokens, with the swap destination being a temporary account B.
3. Jupiter swap 2: The delegate swaps the remaining 5% of the S-tokens into some arbitrary token, with the destination account being their privately owned token account.
4. SwapEnd instruction: Drift pulls the tokens from account B. The Vault's `drift_user` now has $X worth of S-tokens less, but around 0.95*$X worth of T-tokens more. The swap is within the price band and is thus allowed. The reduce-only condition was not violated. The delegate has siphoned off 5% of the swapped funds into their private token account.

The delegate can repeat this until the account is drained.

Method B: Shifting assets to other attacker-controlled users via bad trades

Similarly, the delegate can open orders and trade against them from a private account. This will succeed as long as the orders are within the oracle band. As long as the private account has a price advantage in these swaps, this will result in a de-facto transfer of money from the Vault's `drift_user` account to the attacker's private Drift account.

Note that the orders have to be reduce-only. However, the delegate can siphon off parts of the amounts when swapping base assets into quote.

Location

[1] https://github.com/drift-labs/drift-vaults/blob/6588d75558d953992edcf052ab591ece37d99a94/programs/drift_vaults/src/instructions/liquidate.rs#L60

[2] https://github.com/drift-labs/protocol-v2/blob/a2396c13e57c70c4dca8370316ea0669510dd6ee/programs/drift/src/math/orders.rs#L472

Mitigation Suggestion

There are two main ways to mitigate this:

1. As a short-term "fix", you'll probably have to assume the manager is trusted and only allow a trusted delegate to liquidate.
2. As a long-term fix which should be done anyway but is hard, fix the main contract's delegate functionality.

Remediation

The team implemented the suggested short-term "fix" by introducing the PubKey `4wbNjWbj3kPDbyKnSq8SXVEtAJw4uzE8mJ2QwuK1BCYZ` as a permissioned liquidator. This change was introduced with the commit c97384df7b453d8fea4d4f3a85d154b8a327c125.

[ND-DV-MD-01] Incorrect accounting in withdraw request cancellation instructions

Severity	Impact	Affected Component	Status
MEDIUM	Accounting Error	Accounting	Resolved

Accounting during the cancellation of a withdraw request is done incorrectly.

Normally, shares are tracked as follows:

- The total number of shares is stored in `vault.total_shares`
- The sum of all user-controlled shares is stored in `vault.user_shares`
- Protocol shares are stored in `vault_protocol.protocol_profit_and_fee_shares`
- Manager shares are the shares which are neither user shares nor protocol shares (i.e. `manager_shares = vault.total_shares - vault.user_shares - vault_protocol.protocol_profit_and_fee_shares` [1]

In `CancelWithdrawRequest`, any lost shares are deducted from the user's account, from `vault.total_shares`, `vault.user_shares` and `vault_protocol.protocol_profit_and_fee_shares` (if a vault protocol is present) [2].

Deducting from `vault.user_shares` and `vault_protocol.protocol_profit_and_fee_shares` gives the manager more shares each time a withdrawal is canceled and the vault turned a per-share profit while the withdrawal was pending.

Note that similar accounting issues exist in the two other Withdraw Request Cancellation instructions:

`ManagerCancelWithdrawRequest` deducts the lost shares from `total_shares`, `user_shares`, and `protocol_profit_and_fee_shares`. This means that when a manager cancels a withdrawal request they get more shares than they previously had instead of having less.

`ProtocolCancelWithdrawRequest` deducts the lost shares correctly from `protocol_profit_and_fee_shares` but also from `user_shares`. This results again in the manager getting more shares than they should.

Location

[1] https://github.com/drift-labs/drift-vaults/blob/6588d75558d953992edcf052ab591ece37d99a94/programs/drift_vaults/src/state/vault.rs#L330

[2]

- https://github.com/drift-labs/drift-vaults/blob/e203036f416c5f2a72106910be9868b735acbe0f/programs/drift_vaults/src/state/vault_depositor.rs#L469-L481
- https://github.com/drift-labs/drift-vaults/blob/e203036f416c5f2a72106910be9868b735acbe0f/programs/drift_vaults/src/state/vault.rs#L607-L619

- https://github.com/drift-labs/drift-vaults/blob/e203036f416c5f2a72106910be9868b735acbe0f/programs/drift_vaults/src/state/vault.rs#L896-L909

Relevant Code

```
pub fn cancel_withdraw_request(
        &mut self,
        vault_equity: u64,
        vault: &mut Vault,
        vault_protocol: &mut Option<RefMut<VaultProtocol>>,
        now: i64,
    ) -> Result<()> {
        self.apply_rebase(vault, vault_protocol, vault_equity)?;

        let vault_shares_before: u128 = self.checked_vault_shares(vault)?;
        let total_vault_shares_before = vault.total_shares;
        let user_vault_shares_before = vault.user_shares;
        let protocol_shares_before = vault.get_protocol_shares(vault_protocol);

        let VaultFee {
            management_fee_payment,
            management_fee_shares,
            protocol_fee_payment,
            protocol_fee_shares,
        } = vault.apply_fee(vault_protocol, vault_equity, now)?;

        let vault_shares_lost = self
            .last_withdraw_request
            .calculate_shares_lost(vault, vault_equity)?;
        self.decrease_vault_shares(vault_shares_lost, vault)?;

        vault.total_shares = vault.total_shares.safe_sub(vault_shares_lost)?;
        vault.user_shares = vault.user_shares.safe_sub(vault_shares_lost)?;

        if let Some(vp) = vault_protocol {
            vp.protocol_profit_and_fee_shares = vp
                .protocol_profit_and_fee_shares
                .safe_sub(vault_shares_lost)?;
        }

        [...]

        Ok(())
    }
```

Resolution

The team fixed the wrong accounting within the aforementioned instructions instructions with commits aaff2898dfa2522e316d1d1bd298c12c6dd05a12 and 959457cd1a6790f8cb78a440a7c5cb898f05cc06.

[ND-DV-IN-01] Incorrect account check comments

Severity	Impact	Affected Component	Status
INFORMATIONAL			Resolved

“CHECK: “ comments usually mark accounts that are not sufficiently checked via anchor account checks, e.g. if AccountInfo is used.

Many of these comments are incorrect in the contract. In the following instructions, accounts are marked as “CHECK: checked in drift cpi” even though no such CPI takes place. They are still checked correctly, however not in the way that the comment indicates.

- ApplyRebaseTokenizedDepositor
- ApplyRebase
- CancelWithdrawRequest
- ManagerCancelWithdrawRequest
- ManagerRequestWithdraw
- ProtocolCancelWithdrawRequest
- ProtocolRequestWithdraw
- RedeemTokens
- RequestWithdraw

No impact, however it could lead to false security assumptions in the future.

Relevant Code (Example)

protocol_cancel_withdraw_request.rs, line 59

```
#[derive(Accounts)]
pub struct ProtocolCancelWithdrawRequest<'info> {
    #[account(
        mut,
        constraint = is_protocol_for_vault(&vault, &vault_protocol, &protocol)?
    )]
    pub vault: AccountLoader<'info, Vault>,
    #[account(
        mut,
        constraint = is_vault_protocol_for_vault(&vault_protocol, &vault)?
    )]
    pub vault_protocol: AccountLoader<'info, VaultProtocol>,
    pub protocol: Signer<'info>,
    #[account(
        constraint = is_user_stats_for_vault(&vault, &drift_user_stats)?
    )]
    /// CHECK: checked in drift cpi
    pub drift_user_stats: AccountInfo<'info>,
    #[account(
```

```
20          constraint = is_user_for_vault(&vault, &drift_user.key())?
21      )]
22      /// CHECK: checked in drift cpi
23      pub drift_user: AccountLoader<'info, User>,
24      /// CHECK: checked in drift cpi
25      pub drift_state: AccountInfo<'info>,
26  }
```

Resolution

The irrelevant comments were removed along with most unused accounts with the commit b364193f0e69fa7dbe881cd46263cc5cab20b8f7.

[ND-DV-IN-02] Unnecessary unconstrained accounts in Request-Withdraw

Severity	Impact	Affected Component	Status
INFORMATIONAL	None	RequestWithdraw	Resolved

In RequestWithdraw, `drift_user_stats`, and `drift_state` are passed but not used. `drift_state` is not constrained and can be any account. If there are no plans to add a drift CPI to this instruction (I don't see why you would), it might make sense to remove these.

No impact except that you can have instruction calls with an arbitrary account which may confuse off-chain parsing (but likely won't).

request_withdraw.rs, lines 72

```
pub fn request_withdraw<'c: 'info, 'info>(
    ctx: Context<'_, '_, 'c, 'info, RequestWithdraw<'info>>,
    withdraw_amount: u64,
    withdraw_unit: WithdrawUnit,
) -> Result<()> {

    // [...] <-- does not contain a drift CPI call

}

#[derive(Accounts)]
pub struct RequestWithdraw<'info> {

    // [...]

    #[account(
        constraint = is_user_stats_for_vault(&vault, &drift_user_stats)?
    )]
    /// CHECK: checked in drift cpi
    pub drift_user_stats: AccountInfo<'info>,
    #[account(
        constraint = is_user_for_vault(&vault, &drift_user.key())?
    )]
    /// CHECK: checked in drift cpi
    pub drift_user: AccountLoader<'info, User>,
    /// CHECK: checked in drift cpi
    pub drift_state: AccountInfo<'info>,
}
```

Resolution

The `drift_state` account has been removed from the relevant instructions with commit b364193f0e69fa7dbe881cd46263cc5cab20b8f7. `drift_user_stats` has been left in place (originally it was removed in the same commit, but was re-added in 347e91e52d8fec47089ffe99a941b28a1cf5c990 for future-proofing the instructions). The comments now also clearly mark the user stats account as unused and only included for future-proofing.

[ND-DV-IN-03] Vault Token Accounts: Naming and Seeds

Severity	Impact	Affected Component	Status
INFORMATIONAL	None	TokenAccounts	Acknowledged

There are three account types which are referred to as “vault token accounts”. The first two share the same seed prefix string, which should be avoided. No collisions can occur due to the size of the seed, however we would recommend changing both the name and seed structure.

1. The token account that temporarily holds funds during deposits and withdrawals, seeded as `[b"vault_token_account", vault.key()]` [1]
2. The token account that temporarily holds funds during IF adding and removing, seeded as `[b"vault_token_account", vault.key(), market_index.to_le_bytes()]` [2]
3. The token account specific to a Tokenized VaultDepositor that receives and immediately burns tokens during RedeemTokens, which is an ATA of the Vault account for the VaultDepositor’s mint. (Note that this ATA has to be initialized separately.) [3]

Location

[1] https://github.com/drift-labs/drift-vaults/blob/e203036f416c5f2a72106910be9868b735acbe0f/programs/drift_vaults/src/instructions/initialize_vault.rs#L95

[2] https://github.com/drift-labs/drift-vaults/blob/e203036f416c5f2a72106910be9868b735acbe0f/programs/drift_vaults/src/instructions/initialize_insurance_fund_stake.rs#L50

[3] https://github.com/drift-labs/drift-vaults/blob/e203036f416c5f2a72106910be9868b735acbe0f/programs/drift_vaults/src/instructions/redeem_tokens.rs#L165

Resolution

The team has opted not to make this change as changing PDA derivation at this point will lead to breaking changes, and the severity of the issue does not warrant this action. Neodyme agrees with this assessment.

A | About Neodyme

Security is difficult.

To understand and break complex things, you need a certain type of people. People who thrive in complexity, who love to play around with code, and who don't stop exploring until they fully understand every aspect of it. That's us.

Our team never outsources audits. Having found over 80 High or Critical bugs in Solana's core code itself, we believe that Neodyme hosts the most qualified auditors for Solana programs. We've also found and disclosed critical vulnerabilities in many of Solana's top projects and have responsibly disclosed issues that could have resulted in the theft of over $10B in TVL on the Solana blockchain.

All of our team members have a background in competitive hacking. During such hacking competitions, called CTFs, we competed and collaborated while finding vulnerabilities, breaking encryption, reverse engineering complicated algorithms, and much more. Through the years, many of our team members have won national and international hacking competitions, and keep ranking highly among some of the hardest CTF events worldwide. In 2020, some of our members started experimenting with validators and became active members in the early Solana community. With the prospect of an interesting technical challenge and bug bounties, they quickly encouraged others from our CTF team to look for security issues in Solana. The result was so successful that after reporting several bugs, in 2021, the Solana Foundation contracted us for source code auditing. As a result, Neodyme was born.



B | Methodology

We are not checklist auditors.

In fact, we pride ourselves on that. We adapt our approach to each audit, investing considerable time into understanding the program upfront and exploring its expected behavior, edge cases, invariants, and ways in which the latter could be violated.

We use our uniquely deep knowledge of Solana internals, and our years-long experience in auditing Solana programs to find bugs that others miss. We often extend our audit to cover off-chain components in order to see how users could be tricked or the contract affected by bugs in those components.

Nonetheless, we also have a list of common vulnerability classes, which we always exhaustively look for. We provide a sample of this list here.

Select Common Vulnerabilities

Our most common findings are still specific to Solana itself. Among these are vulnerabilities such as the ones listed below:

- Insufficient validation, such as:
  - Missing ownership checks
  - Missing signer checks
  - Signed invocation of unverified programs
  - Account confusions
  - Missing freeze authority checks
  - Insufficient SPL account verification
  - Dangerous user-controlled bumps
  - Insufficient Anchor account linkage
- Account reinitialization vulnerabilities
- Account creation DoS
- Redeployment with cross-instance confusion
- Missing rent exemption assertion
- Casting truncation
- Arithmetic over- or underflows
- Numerical precision and rounding errors
- Anchor pitfalls, such as accounts not being reloaded
- Non-unique seeds
- Issues arising from CPI recursion
- Log truncation vulnerabilities
- Vulnerabilities specific to integration of Token Extensions, for example unexpected external token hook calls

Apart from such Solana-specific findings, some of the most common vulnerabilities relate to the general logical structure of the contract. Specifically, such findings may be:

- Errors in business logic
- Mismatches between contract logic and project specifications
- General denial-of-service attacks
- Sybil attacks
- Incorrect usage of on-chain randomness
- Contract-specific low-level vulnerabilities, such as incorrect account memory management
- Vulnerability to economic attacks
- Allowing front-running or sandwiching attacks

Miscellaneous other findings are also routinely checked for, among them:

- Unsafe design decisions that might lead to vulnerabilities being introduced in the future
    - Additionally, any findings related to code consistency and cleanliness
- Rug pull mechanisms or hidden backdoors

Often, we also examine the authority structure of a contract, investigating their security as well as the impact on contract operations should they be compromised.

Over the years, we have found hundreds of high and critical severity findings, many of which are highly nontrivial and do not fall into the strict categories above. This is why our approach has always gone way beyond simply checking for common vulnerabilities. We believe that the only way to truly secure a program is a deep and tailored exploration that covers all aspects of a program, from small low-level bugs to complex logical vulnerabilities.

C | Vulnerability Severity Rating

We use the following guideline to classify the severity of vulnerabilities. Note that we assess each vulnerability on an individual basis and may deviate from these guidelines in cases where it is well-founded. In such cases, we always provide an explanation.

Severity	Description
CRITICAL	Vulnerabilities that will likely cause loss of funds. An attacker can trigger them with little or no preparation, or they are expected to happen accidentally. Effects are difficult to undo after they are detected.
HIGH	Bugs that can be used to set up loss-of-funds exploits in a more limited capacity or to render the contract unusable.
MEDIUM	Bugs that do not cause direct loss of funds but that may lead to other exploitable mechanisms or that could be exploited to render the contract partially unusable.
LOW	Bugs that do not have a significant immediate impact and could be fixed easily after detection.
INFORMATIONAL	Bugs or inconsistencies that have little to no security impact but are still noteworthy.

Additionally, we often provide the client with a list of nit-picks, i.e. findings whose severity lies below Informational. In general, these findings are not part of the report.

Neodyme AG

Dirnismaning 55
Halle 13
85748 Garching
Germany

E-Mail: contact@neodyme.io

https://neodyme.io